

Exploitation d'une BD – TP3

Corrigé TP3

Gestion des valeurs NULL

Dans une requête `SELECT ... FROM ... WHERE ...` seuls les tuples pour lesquels la condition figurant dans le `WHERE` est `TRUE` seront affichés.

Remarque n°1.

Les conditions ci-dessous ne sont jamais `TRUE`. Elles renvoient `NULL` :

<code>NULL = ...</code>	<code>NULL > ...</code>	<code>NULL >= ...</code>	<code>NULL like ...</code>
<code>NULL <> ...</code>	<code>NULL < ...</code>	<code>NULL <= ...</code>	<code>NULL not like ...</code>

Exemple

- `(NULL = "ABE")` vaut `NULL`
- `(NULL <> NULL)` vaut `NULL`
- `(NULL NOT LIKE "S1%")` vaut `NULL`

1. Que va retourner la requête suivante ?

```
SELECT *
FROM planning
WHERE prof <> "JBO";
```

planning

id	prof	groupe
1	ABA	S1G1
2	EDU	S1G2
3	JBO	S1G1
4	NULL	S1G3
5	JBO	S1

	id	prof	groupe
▶	1	ABA	S1G1
	2	EDU	S1G2
•	NULL	NULL	NULL

Remarque n°2.

Pour savoir si ... est `NULL`, on écrit la condition : `... is NULL`

Pour savoir si ... n'est pas `NULL` on écrit la condition : `... is not NULL`

Exemple

- `(NULL is NULL)` est `TRUE` ; en SQL, seules les valeurs `NULL` sont considérées comme `NULL`.
- `(NULL is not NULL)` est `FALSE`
- `("JBO" is NULL)` est `FALSE` ; "JBO" est une chaîne de caractères non vide.
- `("JBO" is not NULL)` est `TRUE`

2. Que va retourner la requête suivante :

```
SELECT *
FROM planning
WHERE prof is not NULL ;
```

	id	prof	groupe
▶	1	ABA	S1G1
	2	EDU	S1G2
	3	JBO	S1G1
	5	JBO	S1

Remarque n°3.

Le résultat d'une opération arithmétique contenant NULL est NULL.

De nombreuses fonctions retournent NULL lorsqu'elles reçoivent en paramètre la valeur NULL.

Exemple

- CONCAT("bonjour", NULL, "au revoir") retourne NULL
- LEFT(NULL, 3) retourne NULL

3. Que vont retourner les requêtes suivantes :

- a. SELECT Nom, HTML+BD "Somme"
FROM notes ;

	Nom	Somme
▶	Bob	NULL
	Lea	24

notes

Nom	HTML	BD
Bob	16	NULL
Lea	10	14

- b. SELECT Nom, CONCAT(HTML," et ", BD) "Total"
FROM notes ;

	Nom	Total
▶	Bob	NULL
	Lea	10 et 14

Remarque n°4.

Les fonctions de groupe SUM(), AVG(), COUNT(), MIN(), MAX(), GROUP_CONCAT, etc. ignorent les valeurs NULL.

4. Que vont retourner les requêtes suivantes :

- a. SELECT AVG(HTML) moyHTML, AVG(BD) moyBD
FROM notes ;

	moyHTML	moyBD
▶	13.0000	14.0000

essai

Nom	desc1	desc2
Bob	a	e
Bob	b	NULL
Joe	c	f
Bob	d	g

- b. SELECT nom, GROUP_CONCAT(desc2) "Resume"
FROM essai
GROUP BY nom ;

	nom	Resume
▶	Bob	e,g
	Joe	f

Remarque n°5.

IFNULL(a, b) renvoie **b** si **a** vaut NULL.

NULLIF(a, b) renvoie NULL si **a** vaut **b**.

5. Que va retourner la requête suivante :

```
SELECT *
FROM rapport
WHERE a / NULLIF(b,0) > 1
```

rapport

Nom	a	b
Bob	3	2
Joe	2	0
Lea	4	NULL

⇒ NULLIF(b, 0) :

- renvoie NULL si b = 0, sinon elle renvoie b.
- Cela évite une division par zéro, qui causerait une erreur en SQL.

⇒ a / NULLIF(b,0) > 1 :

- Effectue la division a / b sauf si b est 0 (dans ce cas, NULL est retourné).
- Afficher uniquement les résultats où la division est strictement supérieure à 1.
- Les lignes où b est 0 ou NULL sont ignorées, car toute opération avec NULL donne NULL et NULL > 1 n'est jamais vrai.

	Nom	a	b
▶	Bob	3	2

Remarque n°6.

NULL correspond à une valeur non renseignée ou inconnue.

On peut définir la table de vérité de AND, OR et NOT dans le cas d'une logique ternaire.

6. Complétez intuitivement les cases blanches dans table de vérité ci-contre.

A	B	A OR B	A AND B	NOT A
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	NULL	TRUE	NULL	FALSE
TRUE	FALSE	TRUE	FALSE	FALSE
NULL	TRUE	TRUE	NULL	NULL
NULL	NULL	NULL	NULL	NULL
NULL	FALSE	NULL	FALSE	NULL
FALSE	TRUE	TRUE	FALSE	TRUE
FALSE	NULL	NULL	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE	TRUE

A	B	A OR B	A AND B	NOT A
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	NULL			FALSE
TRUE	FALSE	TRUE	FALSE	FALSE
NULL	TRUE			NULL
NULL	NULL	NULL	NULL	NULL
NULL	FALSE			NULL
FALSE	TRUE	TRUE	FALSE	TRUE
FALSE	NULL			TRUE
FALSE	FALSE	FALSE	FALSE	TRUE

7. On considère la requête suivante sur la table *lesPlannings*:

```
SELECT *
FROM lesPlannings
```

WHERE idgroupe <> "S1G2" AND NOT(a_eu_lieu);

lesPlannings

idseance	idgroupe	idprof	a_eu_lieu
1	S1G1	BEF	TRUE
2	S1G2	ABE	TRUE
3	S1G1	JBO	FALSE
4	S1G3	EDU	NULL
5	S1	BEF	TRUE

idgroupe<>"S1G2"	not(a_eu_lieu)	idgroupe<>"S1G2" AND NOT(a_eu_lieu)
TRUE	FALSE	FALSE

Dites intuitivement ce que retourne la requête précédente, puis vérifiez rigoureusement en complétant la table ci-dessus.

La ligne 3 uniquement.

planning						
idseance	idgroupe	idprof	a_eu_lieu	idgroupe<>"S1G2"	not(a_eu_lieu)	idgroupe<>"S1G2" AND NOT(a_eu_lieu)
1	S1G1	BEF	TRUE	TRUE	FALSE	FALSE
2	S1G2	ABE	TRUE	FALSE	FALSE	FALSE
3	S1G1	JBO	FALSE	TRUE	TRUE	TRUE
4	S1G3	EDU	NULL	TRUE	NULL	NULL
5	S1	BEF	TRUE	TRUE	FALSE	FALSE

Remarque n°7.

prix in (20, NULL, 30) est équivalent à : (prix=20) OR (prix=NULL) or (prix=30)

8. Que vaut 40 in (20, NULL, 30) ?

NULL car (40=20) OR (40=NULL) OR (40=30) renvoie NULL

9. Que vaut 20 in (20, NULL, 30) ?

TRUE car (20=20) OR (20=NULL) OR (20=30) renvoie TRUE

10. Que vaut NULL in (20, NULL, 30) ?

NULL car (NULL=20) OR (NULL=NULL) OR (NULL=30) renvoie NULL

Remarque n°8.

... NOT IN (...) est équivalent à : NOT(... IN (...))

Autrement dit :

prix not in (10,20,30) est équivalent à : not(prix in (10,20,30))

est équivalent à : not((prix=10) OR (prix=20) OR (prix=30))

est équivalent à : (prix<>10) AND (prix<>20) AND (prix<>30)

11. Que vaut 40 not in (20, NULL) ?

NULL car $(40 \diamond 20) \text{ AND } (40 \diamond \text{NULL})$ vaut NULL

12. Que vaut **20 not in (20, NULL)** ?

FALSE car $(20 \diamond 20) \text{ AND } (20 \diamond \text{NULL})$ vaut FALSE

13. Que vaut **NULL not in (20, NULL)** ?

NULL car $(\text{NULL} \diamond 20) \text{ AND } (\text{NULL} \diamond \text{NULL})$ vaut NULL

14. Que vont retourner les requêtes suivantes avec les tables **planning** et **Professeurs** ci-contre :

Professeurs

id	prof	âge
10	ABA	51
11	EDU	43
12	JBO	45
13	KB	35

planning

id	prof	groupe
1	ABA	S1G1
2	EDU	S1G2
3	JBO	S1G1
4	NULL	S1G3
5	JBO	S1

a. select *
from Professeurs
where prof in (select prof
from planning
where id > 3) ;

id	prof	age
12	JBO	45
NULL	NULL	NULL

b. select *
from Professeurs
where prof not in (select prof
from planning) ;

id	prof	age
NULL	NULL	NULL

c. select *
from Professeurs
where prof not in (select prof
from planning
where groupe <> "S1G3") ;

id	prof	age
13	KB	35
NULL	NULL	NULL

Nous utiliserons la base de données **EDTS2** pour les requêtes suivantes :

15. Il existe des profs qui n'ont été affectés à aucune séance.

a. Pourtant, la requête ci-dessous ne retourne rien. Dites pourquoi.

```
select idprof
from Professeurs
where idProf not in
(select distinct idprof from Seances );
```

Car certains enregistrements de la table Seances ont l'attribut idprof à NULL. Autrement dit, la sous-requête est une liste qui contient des NULL.

- b. Corrigez de 2 manières cette requête pour qu'elle retourne les idprof des profs qui n'ont été affectés à aucune séance.

```
select idprof from Professeurs
except
select idprof from Seances;
```

```
select idprof from Professeurs where idProf not in (select distinct idprof from Seances
where idprof is not null) ;
```

16. Écrivez une requête permettant d'afficher le titre, le module et le professeur de toutes les séances. Pour les séances dont le professeur ne peut être connu, on aimerait afficher "Séance en autonomie" dans la colonne professeur.

```
SELECT idSeance, titreSeance, ifnull(idProf, "Autonomie") AS "Professeur"
FROM Seances ;
```

17. Ecrire une requête pour afficher le titre de la séance, le nom du module concerné et le professeur (prénom nom) qui l'anime. On souhaite afficher « Séance en autonomie » pour les séances sans professeur.

```
SELECT distinct s.titreSeance, m.nomModule,
                IFNULL(CONCAT(p.prenomProf, ' ', p.nomProf), 'Séance en autonomie') AS professeur
FROM Seances s
LEFT JOIN Modules m
ON s.idModule = m.idModule
LEFT JOIN Professeurs p
ON s.idProf = p.idProf;
```

18. Ecrire une requête permettant d'afficher le nombre total d'heures des modules qui représente la somme des heures de CM, TD et TP. On souhaite afficher "Apprentissage libre" pour les modules dont ce nombre n'est pas connu.

```
SELECT idModule, IFNULL(CM+TD+TP, "Apprentissage libre") As "Total des heures"
FROM Modules;
```